Proiectare Logică

I CB: Prof. dr. ing. Mariana Mocanu

- mariana.mocanu@cs.pub.ro

I CA & CD: Conf. dr. ing. Costin Chiru

- costin.chiru@cs.pub.ro

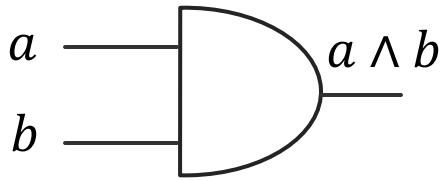
I CC: Prof. dr. ing. Anca Morar

- anca.morar@cs.pub.ro

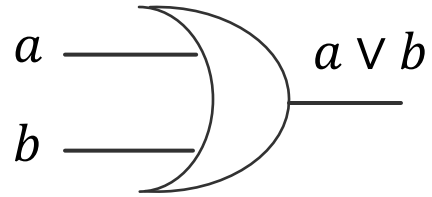
Curs 2: Algebra booleană. Constante booleene, variabile booleene și funcții booleene. Seturi complete de operatori

Fundamentele matematice ale proiectării logice – Algebra Booleană

- O algebră booleană se definește ca un triplet $(B, \wedge, \vee)$, unde B este o mulțime de valori binare, iar $\wedge$ (și), $\vee$ (sau) sunt operatori binari pe B astfel încât să fie adevărate următoarele proprietăți:
  - **P_1**: Pe B, $\wedge$ și $\vee$ sunt comutative
    - $\forall\, a, b \in B$: $a \wedge b = b \wedge a$ și $a \vee b = b \vee a$



AND — Circuit "ȘI" $(\wedge, \cdot)$



OR — Circuit "SAU" $(\vee, +)$

Fundamentele matematice ale proiectării logice – Algebra Booleană

- **P_2**: În B există elemente distincte de identitate 1 și 0 relative la ∧, respectiv ∨
  - $\forall a \in B: a \wedge 1 = a$ *și* $a \vee 0 = a$
- **P_3**: ∧ și ∨ sunt distributive unul față de altul
  - $\forall a, b, c \in B: a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$
  - $\forall a, b, c \in B: a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$
- **P_4**: Pentru oricare $a \in B$ există $\bar{a}$, astfel încât
  - $a \wedge \bar{a} = 0$ *și*
  - $a \vee \bar{a} = 1$

- **P_1-P_4 sunt considerate AXIOMELE algebrei booleene**

Algebra Booleană

- Algebra booleană nu e decât o particularizare peste B relativ la proprietățile definite
- Unele concepte sunt cunoscute de la programare:
  - Tipuri de date: numerice, logice, text
  - Operatori: numerici, logici, relaționali
- Diferențe/asemănări între operatori logici și relaționali: cei relaționali operează pe date numerice și întorc rezultate logice

Teoremele algebrei booleene (1)

- **T_1**: Elementul $\bar{a}$ asociat cu un element $a \in B$ prin proprietatea P_4 este unic.
  - Demonstratie: prin reducere la absurd (demonstrația matematică)
    - Presupunem că $\exists\, x, y \in B$ a.î.
      - $a \wedge x = 0, a \vee x = 1$
      - $a \wedge y = 0, a \vee y = 1$
    - Utilizând $P_2 \Rightarrow x = y$
  - Altă demonstrație: metoda verificării comportamentului → nu e metodă matematică, dar e ușor de adaptat la PL (prin tabela de adevăr)

În curs sunt foarte puține lucruri care trebuie memorate, restul se pot deduce una din alta! → Nu e important numărul teoremei, ci să știți să o aplicați!

Teoremele algebrei booleene (2)

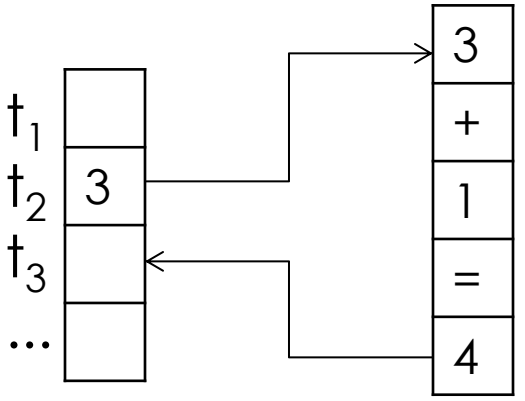
- **T_2**: Fie $a \in B$; atunci $a \vee a = a$ *și* $a \wedge a = a$
- **T_3**: Fie $a \in B$; atunci $a \vee 1 = 1$ *și* $a \wedge 0 = 0$
- **T_4**: Legea de absorbție:
  - $a \vee (a \wedge b) = a$
  - $a \wedge (a \vee b) = a$
- **T_5**: $\vee$ şi $\wedge$ sunt asociative:
  - $a \wedge (b \wedge c) = (a \wedge b) \wedge c$
  - $a \vee (b \vee c) = (a \vee b) \vee c$
- **T_6**: Principiul dublei negări: $\forall a \in B, \bar{\bar{a}} = a$

Teoremele algebrei booleene (3)

- **T_7**: $\bar{0} = 1 \; \text{și} \; \bar{1} = 0$
- **T_8**: Regulile lui de Morgan:
  - $\overline{a \wedge b} = \bar{a} \vee \bar{b}$
  - $\overline{a \vee b} = \bar{a} \wedge \bar{b}$
- Generalizare de Morgan: Fie $x_1, x_2, \ldots, x_n \in B$
  - $\overline{x_1 \wedge x_2 \wedge \cdots \wedge x_n} = \overline{x_1} \vee \overline{x_2} \vee \cdots \vee \overline{x_n}$
  - $\overline{x_1 \vee x_2 \vee \cdots \vee x_n} = \overline{x_1} \wedge \overline{x_2} \wedge \cdots \wedge \overline{x_n}$

Convenție

- $\bar{} = !,\ \vee = +\ iar\ \wedge = \cdot$
- Observație: sunt operatori logici nu matematici
  - Ex: $1 + 1 = 1$
- Unde ați mai întâlnit diferențe față de matematică?
  - Incrementare: $x = x + 1$
  - De ce e corect? Are conotație temporală



La ce sunt utile teoremele?

- 2 aspecte:
  - Matematic: expresii, funcții, reducerea (**minimizarea**) acestora
  - Practic: asociem fiecărei funcții un anumit circuit → punând laolaltă diverse dispozitive în diferite feluri putem genera diverse funcționalități, dar e important câte astfel de dispozitive se folosesc, mai ales având în vedere T_2,T_4 și T_6

Constante, variabile, funcții booleene

- Într-o algebră booleană, 0 și 1 sunt singurele constante.
- O variabilă logică este o mărime logică ce ia o valoare ce se poate schimba.
- O expresie logică este o succesiune de variabile/constante unite prin operatori, respectând regulile de evaluare ale operatorilor
  - $a, b \epsilon B$, $ab+$ nu e expresie, $a+b$ e expresie
- O funcție booleană de n variabile $x_1, x_2, \dots, x_n$, notată $f(x_1, x_2, \dots, x_n)$ se definește ca o aplicație a lui B^n $(B \times B \times \cdots \times B) \rightarrow B$. Variabilele $x_1, x_2, \dots, x_n$ sunt independente
- O funcție booleană = rezultatul oricărei combinații finite de constante și variabile realizate cu ajutorul operatorilor !, ·, +

Forme de reprezentare a funcțiilor booleene

- 1. Algebrică: se definește funcția pe baza unei expresii:
  - Teoremele învățate ne ajută la reprezentarea algebrică $f(x_1, x_2, \dots, x_n) = expresie$
  - Ex: $f(a, b) = a + b$
  - Expresiile pot fi combinate → forme relativ complicate
- 2. Tabel de adevăr
  - (-) Foarte multe combinații dacă funcția depinde de multe variabile
  - (+) Se cunoaște rezultatul pentru fiecare combinație în parte
- 3. Grafică – diagrame Karnaugh
- 4. Zecimală – asociază fiecărui monom o valoare zecimală și permite automatizarea minimizării

Reguli de precedență a operatorilor

- Evaluarea unei expresii se face respectând anumite reguli (reguli de precedență a operatorilor):
  - 1. se evaluează, mai întâi, toate subexpresiile conținute în paranteze, evaluarea pornind de la zona cea mai imbricată (folosind regulile de mai jos) : $a \cdot (!\, b + (a \cdot c))$
  - 2. se aplică toți operatorii NEGAȚIE (NOT, !, $\bar{}$): $a + b + c \cdot !\, b + !\, c$
  - 3. Se aplică toți operatorii ȘI ($\wedge$,$\cdot$): $a + b \cdot c$
  - 4. Se aplică toți operatorii SAU (v, +)
  - 5. Evaluarea se face de la stânga la dreapta dacă avem același operator peste tot

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Redă într-o formă ordonată funcționalitatea unei funcții pentru fiecare combinație de intrare posibilă (2^N combinații)

x_1	x_2	…	x_n	f
0	0	…	0	$f(0, 0, \dots 0)$
0	0	…	1	$f(0, 0, \dots 1)$
1	1	…	1	$f(1, 1, \dots 1)$

- Pe fiecare linie există o singură valoare logică

- De exemplu, $f_1(a) = \bar{a}$

a	$\bar{a}$
0	1
1	0

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Alte exemple:
  - $f_2 = a \wedge b = a \cdot b$ (AND)
  - $f_3 = a \vee b = a + b$ (OR)

a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

- De unde știm că aceste valori sunt așa și nu altfel?
  - Din $P_{1\text{-}4}$ și din teoreme

Reprezentarea funcțiilor booleene prin tabele de adevăr

- Alte exemple:
- $f(a,b,c) = \bar{a}b\bar{c} + ab\bar{c} + \bar{a}c + bc$

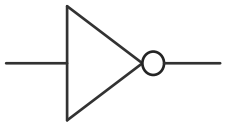
a	*b*	*c*	*f*

- Cum facem?
  - Scriem toate combinațiile de valori ordonate
    - Atenție la simetrie!
  - Evaluăm fiecare combinație în parte

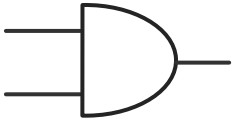
Dispozitive

- Există diverse dispozitive (elemente de comutație) ce generează rezultatele funcțiilor f_1 - f_3

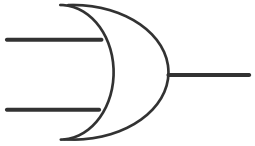
$f_1(a) = \bar{a}$



$f_2 = a \cdot b$ (AND)



$f_3 = a + b$ (OR)



- Pe unde vedeți ∘, înseamnă că ieșirea e negată!

a	$\bar{a}$
0	1
1	0

a	b	$a \wedge b$	$a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Teoremele algebrei booleene (4)

- Există operatori fundamentali și operatori derivați (care se exprimă cu ajutorul celor fundamentali):
- Se definește operatorul relațional „→" a.î. pentru fiecare pereche de elemente $a, b \in B$, $a \to b$ este echivalent cu $a \cdot \bar{b} = 0$.
- **T_9**: Relația „→" este reflexivă: $a \to a, \ \forall a \in B$
- **T_{10}**: Relația „→" este tranzitivă: dacă $a \to b$ și $b \to c$, atunci $a \to c$
- **T_{11}**: Dacă $x \to y$ şi $x \to z$, atunci $x \to y \cdot z$
- **T_{12}**: Dacă $x \to y$, atunci $x \to y + z$, $\forall z \in B$
- **T_{13}**: Dacă $\bar{x} \to \bar{y}$, atunci $y \to x$
- **T_{14}**: Dacă $x \to y$ şi $y \to x$, atunci $x = y$

Monom

- Un monom de n variabile $x_1, x_2, \ldots, x_n$ este o expresie de forma ${x_1}^{e_1} \cdot {x_2}^{e_2} \cdot \ldots \cdot {x_n}^{e_n}$, unde

$${x_i}^{e_i} = \begin{cases} x_i, & dacă \quad e_i = 1 \\ !\, x_i, & dacă \quad e_i = 0 \end{cases}$$

- Ex: având variabilele $a,b,c,$ se pot crea $2^3 = 8$ monoame diferite: $\bar{a}\bar{b}\bar{c}$, $\bar{a}\bar{b}c$, $\bar{a}b\bar{c}$, $\bar{a}bc$, $a\bar{b}\bar{c}$, $a\bar{b}c$, $ab\bar{c}$ și abc

Forma Disjunctivă

- Reprezentarea funcției sub formă de monoame conduce la forma disjunctivă a funcției
- Ex: $f_6 = ab + a\bar{b}$ - funcție disjunctivă ce depinde de 2 variabile (a, b)
- Dar $f_6 = \overset{P_3}{a} \cdot \overset{P_4}{(b + \bar{b})} = a \overset{P_2}{\cdot 1} = a$
- Câte circuite sunt necesare pentru implementare:
  - a) în forma inițială? 1 NOT, 2 ȘI, 1 SAU
  - b) în forma finală? Nimic
- Iată de ce sunt utile teoremele și proprietățile!

Forma Disjunctivă Normală (FDN)

- $f_7(a,b,c) = ab + a\bar{b}c$ → uneori ne ajută ca funcțiile să fie într-o formă omogenă → FDN
- FDN cuprinde monoame ce depind de toate variabilele de care depinde funcția
- Monoamele ce depind de toate variabilele de care depinde funcția (adică sunt complete) se numesc mintermi.
- $ab = ab \cdot 1 = ab\,(c + \bar{c}) = abc + ab\bar{c}$
- $f_7(a,b,c) = abc + ab\bar{c} + a\bar{b}c$ FDN

Forma Disjunctivă Normală (FDN)

- De ce am face asta? (de ce să complicăm funcțiile)?
  - La unele circuite e posibil să fie mai ușor așa
  - Prin FDN se trece foarte ușor de la tabel de adevăr la forma algebrică și invers

a	b	c	$f_7 = ab + a\bar{b}c$
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

- $f_7(a,b,c) = abc + ab\bar{c} + a\bar{b}c$
- În forma inițială e mai dificil să evaluăm decât în FDN, pentru că $f = 1$ dacă cel puțin unul din mintermeni e 1 → $x + 1 = 1$
- Se poate și invers: se dă tabelul de adevăr și se cere forma algebrică a funcției → FDN cu termenii care au 1 în tabel

Forma Disjunctivă Normală Completă (FDNC)

- Def: Suma tuturor mintermilor ne dă FDNC.
- Ex: $f_8(a,b) = \bar{a}\,\bar{b} + \bar{a}\,b + a\,\bar{b} + ab = 1$
- **T_{15}**: Orice funcție reprezentată ca FDNC are valoarea 1.
  - Dem: recursiv, pentru modul general SAU particular, prin factor comun SAU prin tabel de adevăr – peste tot e 1, deci valoarea e 1
- Două funcții se consideră echivalente dacă iau aceleași valori pentru fiecare combinație de valori ale variabilelor de intrare
- **T_{16}**: Fiecare funcție dintr-o algebră booleană este echivalentă cu o FDN (oricât de complicată e o expresie, ea poate fi adusă la FDN)

Forma Conjunctivă Normală (FCN)

- Suma logică a tuturor variabilelor în forma lor directă sau negată se numește maxterm.
- Ex: $a + b + \bar{c}$
- O funcție este reprezentată în FCN dacă este reprezentată ca produs logic al maxtermilor săi.
- Ex: $f_9(a, b, c) = (a + b)(a + c)$ nu e FCN
  $f_{10}(a, b) = (a + b)(a + \bar{b})$ e FCN
- FDN: sumă de mintermi
- FCN: produs de maxtermi

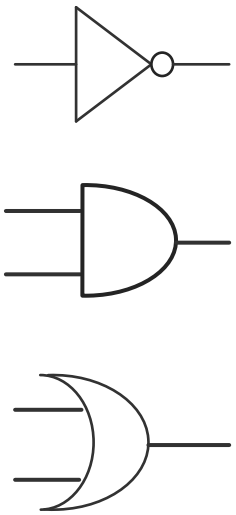
FDN ⇿ FCN

- Pentru a trece dintr-o formă în alta se folosesc regulile lui de Morgan
- Ex: $f_{10}(a,b) = (a+b)\left(a+\bar{b}\right) = \overline{\overline{(a+b)(a+\bar{b})}} = \overline{\bar{a}\bar{b} + \bar{a}b}$

- Se mai poate apela și la desfacerea parantezelor:
- Ex: $f_{10}(a,b) = (a+b)\left(a+\bar{b}\right) = a + a\bar{b} + ab + b\bar{b} = a + a \cdot 1 = a$

Set complet de operatori

- Introducerea de operatori nu se face oricum, ci trebuie să respecte niște reguli.
- Un set de operatori se consideră complet din punct de vedere funcțional, dacă orice funcție booleană poate fi reprezentată prin intermediul acestor operatori.
- Ex: (!, ·, +) = set complet fundamental de operatori.



Seturi complete de operatori

- (!, +) = set complet de operatori: de Morgan

$$a \cdot b = \overline{\bar{a} + \bar{b}}$$

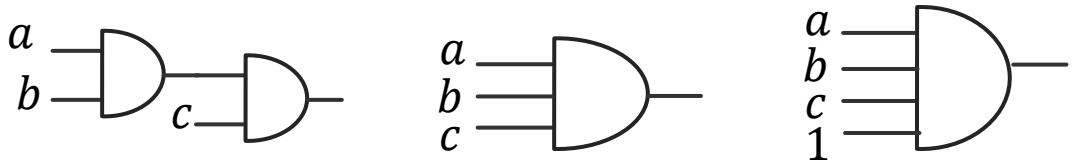
- (!, ·) = set complet de operatori: de Morgan

$$a + b = \overline{\bar{a} \cdot \bar{b}}$$

- Orice funcție poate fi implementată folosind exclusiv



- Observație: porțile pot avea și mai multe intrări, nu fix 2. În practică pot avea 2,3,4 sau 8 intrări.
- Ex: $a \cdot b \cdot c$ cu ȘI cu 2, 3, 4 intrări: $abc = (ab)c$



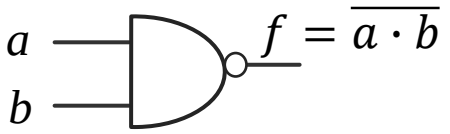
Seturi complete de operatori

- Operator nou: NAND – „ȘI-NU" (↑): $a \uparrow b = !(a \cdot b)$
- Operator nou: NOR – „SAU-NU" (↓): $a \downarrow b = !(a + b)$
- $a \cdot b = !(a \uparrow b)$ → · se poate exprima prin (!, ↑)
- Dar (!, ·) era set complet de operatori → (!, ↑) e set complet de operatori
- Similar, (!, ↓) e set complet de operatori pentru că (!, +) e set complet de operatori, iar $a + b = !(a \downarrow b)$
- $a \uparrow a = !(a \cdot a) = !a$ → ! se poate exprima prin ↑ și cum (!, ↑) e set complet de operatori → (↑) e set complet de operatori
- Similar, $a \downarrow a = !(a + a) = !a$ → (↓) e set complet de operatori

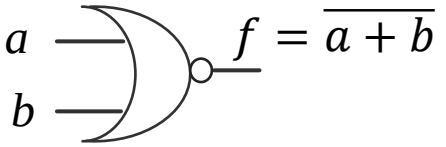
Seturi complete de operatori

- Datorită faptului că NOR și NAND reprezintă seturi complete de operatori, există elemente de comutație pentru ele:

a	b	$a \uparrow b$
0	0	1
0	1	1
1	0	1
1	1	0



a	b	$a \downarrow b$
0	0	1
0	1	0
1	0	0
1	1	0



Seturi complete de operatori

- Investigăm „ →” : $a \to b$ => $a \cdot !\, b = 0$ →
- $a \to b = !\,(a \cdot !\, b)$ → $!\,(a \to b) = a \cdot !\, b$
- Notăm $!\, b = B$ → $a \cdot B = !\,(a \to !\, B)$ → · se poate exprima prin ! Si →
- Cum (!, ·) = set complet de operatori → (!, →) = set complet de operatori
- Concluzie:
  - (!, ·), (↑), (!, +), (↓), (!, →) = seturi complete de operatori
  - (!, ·, +), (!, ↑), (!, ↓) = seturi supracomplete de operatori

